JAVA • CORE CONCEPT NOTES

# Exception Handling

A complete, exam-focused study guide with code, diagrams & revision sheet

### Topics Covered

1. Types of Exceptions
2. `try`, `catch`, `finally`, `throws` keywords
3. Creating Your Own Exception
4. Exceptions and Inheritance

Prepared for quick understanding and last-minute revision

TOPIC 1

# Types of Exceptions

## 1.1 What is an Exception?

An **exception** is an event that disrupts the normal flow of a program's execution during runtime. It is an object that represents an error condition, created when something goes wrong (e.g., dividing by zero, accessing an invalid array index, or a missing file).

Java handles exceptions using an **object-oriented approach** — every exception is an object of a class that ultimately derives from `java.lang.Throwable`.

**THROWABLE CLASS HIERARCHY**

**Throwable**
- ├── **Error** (serious, unrecoverable — e.g. OutOfMemoryError, StackOverflowError)
- └── **Exception**
  - ├── Checked Exceptions (IOException, SQLException...)
  - └── RuntimeException (Unchecked)
    - ├── ArithmeticException
    - ├── NullPointerException
    - ├── ArrayIndexOutOfBoundsException
    - └── ClassCastException

## 1.2 Main Categories

| Category | Description | Examples |
|---|---|---|
| **Checked Exception** | Checked by the compiler at **compile time**. Must be either caught using `try-catch` or declared using `throws`. Represents conditions a well-written program should anticipate (e.g., a file not being found). | `IOException`, `SQLException`, `ClassNotFoundException` |
| **Unchecked Exception** (Runtime Exception) | Not checked at compile time; occurs during **program execution**. Usually caused by programming bugs (bad logic, invalid casting, null access). | `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException` |
| **Error** | Represents serious problems that a normal application should **not try to catch** — usually related to the environment (JVM) running out of resources. | `OutOfMemoryError`, `StackOverflowError` |

### Example: Unchecked Exception

```
public class Demo {
    public static void main(String[] args) {
        int a = 10, b = 0;
        int result = a / b;   // ArithmeticException at runtime
        System.out.println(result);
    }
}
```

### Example: Checked Exception

```
import java.io.*;

public class Demo {
    public static void main(String[] args) throws IOException {
        FileReader fr = new FileReader("data.txt");
        // compiler forces handling: try-catch OR throws
    }
}
```

**EXAM POINT**

Checked exceptions are checked at **compile time**; unchecked exceptions (RuntimeException and its subclasses) are checked at **run time**. Errors are never expected to be caught by the application.

**COMMON MISTAKE**

Thinking all exceptions must be handled with `try-catch`. Only **checked exceptions** force compile-time handling; unchecked exceptions compile fine even if ignored (but crash the program if they occur).

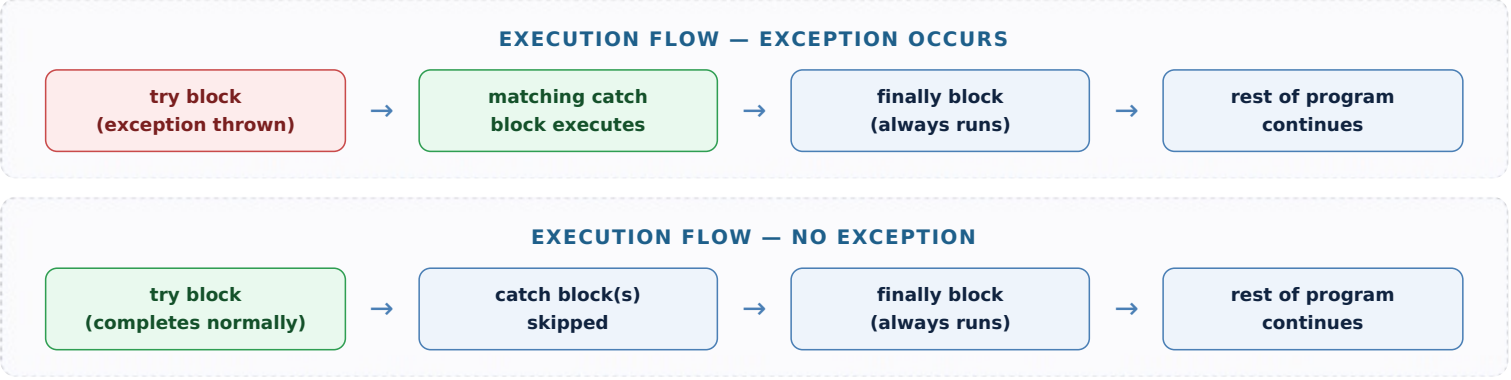
TOPIC 2

# try, catch, finally, throws Keywords

## 2.1 The try-catch Block

The `try` block contains code that might throw an exception. The `catch` block "catches" and handles the exception if it occurs. If no exception occurs, all `catch` blocks are skipped.

```
try {
    // risky code
    int data = 50 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage());
}
```




## 2.2 Multiple catch Blocks

A single `try` can be followed by multiple `catch` blocks to handle different exception types differently. Java checks each `catch` from top to bottom and executes the **first match**.

```
try {
    int[] arr = new int[5];
    arr[10] = 30 / 0;
} catch (ArithmeticException e) {
    System.out.println("Arithmetic problem");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array index problem");
} catch (Exception e) {
    System.out.println("Some other exception");
}
```

**COMMON MISTAKE**

Placing a superclass catch block (e.g. `catch(Exception e)`) **before** a subclass catch block (e.g. `catch(ArithmeticException e)`) causes a **compile-time error** — "unreachable catch block." The more specific exception must always come first.

## 2.3 The finally Block

The `finally` block always executes, whether or not an exception occurs, and whether or not it is caught — it is typically used for cleanup (closing files, releasing resources).

```
try {
    System.out.println("try block");
    int x = 5 / 0;
} catch (ArithmeticException e) {
    System.out.println("catch block");
} finally {
    System.out.println("finally block - always executes");
}
```

**EXAM POINT**

`finally` executes even if the `try` / `catch` block has a `return` statement. The only case it may not run is if `System.exit()` is called or the JVM crashes.

## 2.4 The throws Keyword

`throws` is used in a method signature to **declare** that the method might throw one or more checked exceptions, shifting the responsibility of handling them to the caller (ultimately the **JVM itself**, if no caller in the chain handles it).

```
void readFile() throws IOException {
    FileReader fr = new FileReader("data.txt");
}

public static void main(String[] args) {
    try {
        readFile();
    } catch (IOException e) {
        System.out.println("Handled in caller: " + e.getMessage());
    }
}
```

**REMEMBER**

`main()` is the caller of `readFile()`. If the caller wraps the call in `try-catch` (as above), the **caller handles** the exception — the JVM is never involved. But if the caller has **no try-catch** and just declares `throws` too, there is no one left to pass it to, so it is **handled by the JVM** itself (prints the stack trace and terminates the program).

### throw vs throws

| | `throw` | `throws` |
|---|---|---|
| **Purpose** | Used to actually **throw** an exception object | Used to **declare** possible exceptions |
| **Used** | Inside a method body | In the method signature |
| **Followed by** | A single exception instance | One or more exception class names |
| **Example** | `throw new ArithmeticException("err");` | `void m() throws IOException {...}` |

**SHORTCUT TO REMEMBER**

`throw` = single word → single exception object thrown.
`throws` = plural spelling → can list multiple exception types.

**REMEMBER**

Order of execution: **try → (catch if exception matches) → finally**, always in that sequence.

TOPIC 3

# Creating Your Own Exception

## 3.1 Why Custom Exceptions?

Built-in exceptions cannot always describe application-specific error conditions meaningfully (e.g., "Insufficient Balance" or "Invalid Age"). A **user-defined (custom) exception** lets you create a class with a name and message relevant to your own business logic.

## 3.2 Steps to Create a Custom Exception

1. Create a new class that **extends** `Exception` (for a checked exception) or `RuntimeException` (for an unchecked exception).
2. Provide a constructor that calls `super(message)` to pass the error message to the parent class.
3. `throw` an object of this class wherever the invalid condition occurs.
4. Handle it using `try-catch` (or declare with `throws`) at the calling point.

### Example: Custom Checked Exception

```
// STEP 1: Create a class extending Exception (checked custom exception)
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        // STEP 2: constructor calls super(message) to pass message to parent
        super(message);
    }
}

class BankAccount {
    double balance = 1000;

    void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            // STEP 3: throw an object of this class when the condition is invalid
            throw new InsufficientBalanceException("Balance too low for withdrawal");
        }
        balance -= amount;
        System.out.println("Withdrawal successful. Balance: " + balance);
    }
}

public class Demo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        // STEP 4: handle it using try-catch at the calling point
        try {
            acc.withdraw(5000);
        } catch (InsufficientBalanceException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

### Example: Custom Unchecked Exception

```
// STEP 1: Create a class extending RuntimeException (unchecked custom exception)
class InvalidAgeException extends RuntimeException {
    public InvalidAgeException(String message) {
        // STEP 2: constructor calls super(message) to pass message to parent
        super(message);
    }
}

class Voter {
    void checkAge(int age) {
        if (age < 18) {
            // STEP 3: throw an object of this class when the condition is invalid
            throw new InvalidAgeException("Age must be 18 or above");
        }
        System.out.println("Eligible to vote");
    }
}
```

```
public class Demo {
    public static void main(String[] args) {
        Voter v = new Voter();
        // STEP 4: the caller may use try-catch to handle it,
        // or simply leave it out and let it propagate (since it's unchecked)
        try {
            v.checkAge(15);
        } catch (InvalidAgeException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

**EXAM POINT**

Extending `Exception` creates a **checked** custom exception (must be declared with `throws` or caught). Extending `RuntimeException` creates an **unchecked** custom exception (no compiler enforcement).

**REMEMBER**

Always call `super(message)` inside the constructor so that `getMessage()` works correctly on the custom exception object.

**COMMON MISTAKE**

Forgetting to call `super(message)` — the exception is still thrown, but `e.getMessage()` returns `null` instead of your intended text.

TOPIC 4

# Exceptions and Inheritance

## 4.1 Exceptions Form a Class Hierarchy

Since every exception is a class, and Java classes follow inheritance, exceptions are naturally organized in a **hierarchy** rooted at `Throwable`. This means an exception object "is-a" instance of every one of its superclasses too.

For example, an `ArithmeticException` is also a `RuntimeException`, which is also an `Exception`, which is also a `Throwable`.

**CUSTOM EXCEPTION IN THE HIERARCHY**

**Throwable**
└── **Exception**
└── InsufficientBalanceException (your custom class)

## 4.2 Catching by Superclass Type

Because of inheritance, a `catch` block written for a **superclass** can catch objects of any of its **subclasses** too.

```
try {
    int[] arr = new int[3];
    arr[5] = 10;   // throws ArrayIndexOutOfBoundsException
} catch (RuntimeException e) {     // superclass catches subclass object
    System.out.println("Caught by superclass: " + e);
}

/* Output:
Caught by superclass: java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
*/
```

**EXAM POINT**

`catch(Exception e)` can catch **any** checked or unchecked exception, since `Exception` is the superclass of all of them. It is best practice to catch specific exceptions first and generic ones last.

### Example: Specific Exceptions First, Generic Last

```
try {
    int[] arr = new int[5];
    arr[10] = 30 / 0;
} catch (ArithmeticException e) {        // specific — matches only divide-by-zero etc.
    System.out.println("Arithmetic problem");
} catch (ArrayIndexOutOfBoundsException e) { // specific — matches only bad array index
    System.out.println("Array index problem");
} catch (Exception e) {                  // generic — matches almost ANY exception
    System.out.println("Some other exception");
}

/* Output:
Arithmetic problem
*/
```

## 4.3 Overriding Methods That Throw Exceptions

When a subclass **overrides** a method that is declared with `throws`, Java's inheritance rules restrict what exceptions the overriding method may declare:

- It may declare the **same** checked exception(s) as the parent method.
- It may declare a **subclass** of the exception(s) declared by the parent.
- It may declare **no** exception at all.
- It must **not** declare a new or broader checked exception than the parent method.

```java
class Parent {
    void show() throws IOException {
        System.out.println("Parent method");
    }
}

class Child extends Parent {
    // Allowed: narrower/same exception, or none at all
    void show() throws FileNotFoundException {
        System.out.println("Child method");
    }
}

public class Demo {
    public static void main(String[] args) throws IOException {
        Parent p = new Child();   // upcasting — allowed due to inheritance
        p.show();                   // runs Child's overridden version
    }
}

/* Output:
Child method
*/
```

**COMMON MISTAKE**

Declaring a **broader checked exception** in the overriding method than the parent declares (e.g., parent throws `IOException`, child throws `Exception`) — this causes a **compile-time error**.

## 4.4 Custom Exception Inheriting Behaviour

A custom exception class automatically inherits useful methods from `Throwable` / `Exception`, such as:

| Method | Purpose |
|---|---|
| `getMessage()` | Returns the detail message passed via `super(message)` |
| `toString()` | Returns class name + message as a readable string |
| `printStackTrace()` | Prints the exception type, message, and call stack |
| `getCause()` | Returns the underlying cause (if chained) |

```java
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);   // inherited constructor chain stores the message
    }
}

public class Demo {
    public static void main(String[] args) {
        try {
            throw new InsufficientBalanceException("Balance too low");
        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
            System.out.println(e.toString());
            e.printStackTrace();
        }
    }
}

/* Output:
Balance too low
InsufficientBalanceException: Balance too low
InsufficientBalanceException: Balance too low
    at Demo.main(Demo.java:11)
*/
```

**Explanation:** `InsufficientBalanceException` never defines `getMessage()`, `toString()`, or `printStackTrace()` itself — it inherits all of them from `Exception` / `Throwable` simply by extending the class. Calling `super(message)` in the constructor stores the message inside the inherited `Throwable` object, so `getMessage()` returns it automatically, `toString()` combines it with the class name, and `printStackTrace()` uses it to print the full error trace — all without writing a single extra line of code.

**SHORTCUT TO REMEMBER**

"Narrower or nothing" — an overriding method may throw the **same**, a **subclass**, or **no** checked exception, never a broader one.

**REMEMBER**

Custom exceptions gain the entire `Throwable` toolkit ( `getMessage()` , `printStackTrace()` , etc.) automatically through inheritance — you don't need to rewrite them.

# Quick Revision — Cheat Sheet

**Exception** — event disrupting normal program flow at runtime; an object of a `Throwable` subclass.

**Checked exception** — checked by compiler; must handle with try-catch or throws. e.g. `IOException`.

**Unchecked exception** — RuntimeException family; not compiler-enforced. e.g. `ArithmeticException`.

**Error** — serious JVM-level problem, not meant to be caught. e.g. `StackOverflowError`.

**try** — block containing risky code.

**catch** — handles a specific exception type; most specific type first.

**finally** — always executes (cleanup code), regardless of exception.

**throw** — used to actually throw one exception object.

**throws** — declares possible checked exceptions in method signature.

**Custom exception** — class extending `Exception` (checked) or `RuntimeException` (unchecked); call `super(message)`.

**Hierarchy rule** — a superclass catch block can catch subclass exception objects.

**Override rule** — overriding method: same, narrower, or no checked exception — never broader.

## One-Line Exam Facts

- Root of all exceptions/errors: `java.lang.Throwable`.
- Two direct children of `Throwable`: `Error` and `Exception`.
- A `try` block must be followed by at least one `catch` or a `finally`.
- Execution order: `try` → matching `catch` → `finally`.
- Multiple `catch` blocks: specific exception classes before general ones, or compile error.
- `throw` throws one object; `throws` declares a list of exception types.
- Custom exceptions typically extend `Exception` or `RuntimeException` and call `super(message)`.
- `catch(Exception e)` catches all exception types due to inheritance.

**FINAL SHORTCUT**

Think: **T-C-F-T** → **T**ry risky code, **C**atch specific to general, **F**inally cleans up always, **T**hrows declares upward.